Capitolo 1

Introduzione e Definizione del Problema

L'**anomaly detection** (o outlier detection), è il processo che mira ad identificare i dati che deviano in modo significativo dalla maggioranza delle osservazioni, portando a presupporre che siano generati da un meccanismo differente.

Individuare queste irregolarità comporta sfide uniche dovute alla natura intrinseca del problema. Tra le principali complessità sono presenti:

- l'incognita delle anomalie: le anomalie rimangono sconosciute finché non si verificano.
- l'eterogeneità delle anomalie: le diverse classi di anomalie possono avere caratteristiche totalmente diverse tra loro.
- la rarità delle anomalie: vi è un grande squilibrio di classe rispetto la maggioranza di dati normali. [19]

Il processo di rilevamento delle anomalie ha una lunga storia, costituita da studi e applicazioni di metodologie sempre più innovative che puntano alla risoluzione delle molteplici sfide che caratterizzano la materia. Le principali per le quali le tecnologie moderne, in particolare il deep learning, giocano un ruolo fondamentale sono:

1. **Basso tasso di richiamo (Recall)**: data la rarità e l'eterogeneità delle anomalie, alcuni metodi possono generare dei "falsi positivi". La sfida è quella, quindi, di ridurre i falsi positivi, aumentando le capacità di individuare le vere anomalie.
2. **Dati ad alta dimensionalità e interdipendenti**: le anomalie, evidenti in poche dimensioni, tendono a scomparire all'aumentare della complessità dei dati. Inoltre, risulta sfidante individuare anomalie quando le istanze sono interdipendenti tra loro.

3. **Apprendimento efficiente con pochi dati**: ottenere grandi quantità di dati anomali spesso è costoso e poco pratico. La sfida è riuscire a massimizzare lo sfruttamento dei dati parzialmente etichettati per poter distinguere i dati normali dalle anomalie e quindi generalizzare il modello anche per anomalie mai viste prima.

4. **Resilienza al rumore**: i dati di addestramento reali non sono quasi mai perfettamente puliti, ma anzi spesso contengono etichette errate o anomalie non classificate. Sorge necessario sviluppare modelli capaci di tollerare, o ancor meglio sfruttare questi dati.

5. **Rilevamento di anomalie complesse**: la maggior parte dei sistemi classici è addestrata in modo tale da concentrarsi su singole anomalie puntuali, è necessario espandere il rilevamento ad anomalie contestuali o collettive. La sfida è quella di creare, quindi, modelli capaci di ricordare il contesto e valutare interi gruppi di dati.

6. **Spiegabilità (Anomaly Explanation)**: spesso i modelli complessi agiscono come scatole nere. Oltre a rilevare un'anomalia, il sistema deve fornire delle motivazioni che siano interpretabili. Questa "trasparenza" è fondamentale per garantire affidabilità (soprattutto in domini critici), permettere la supervisione umana e garantire l'ottimizzazione del modello bilanciando potenza e verificabilità.

L'applicazione di metodi basati sul deep learning riesce quindi a superare i limiti dei metodi più tradizionali, ottimizzando il processo interamente e apprendendo rappresentazioni specifiche per il rilevamento di anomalie. Riescono a sfruttare dati parzialmente etichettati, complessi e anche ad alta dimensionalità, unificando il rilevamento e la spiegazione in un unico sistema.

1.1 L'anomaly detection nella Computer Vision

La Visual Anomaly Detection (VAD) è un punto cruciale nel campo del machine learning e della computer vision in quanto l'obiettivo è identificare, all'interno di dati visivi come ad esempio immagini o nuvole di punti, ciò che devia rispetto ad un concept stabilito di "normalità". [6] Rispetto alla maggior parte dei metodi di machine learning classici, che assumono di operare in un sistema statico e chiuso, in cui tutte le classi sono note a priori; la VAD studia come i modelli possano gestire informazioni sconosciute e incerte in un ambiente aperto e dinamico. [26]

A seconda del grado di ispezione, la VAD si divide in **image-level**, stabilendo se l'intera immagine è anomala e **pixel-level**, in cui viene localizzata l'esatta regione del difetto, rivelandosi vitale in campi come l'ispezione di difetti industriali, identificazione di lesioni mediche e rilevamento di ostacoli nella guida autonoma. [26][19].

1.1.1 Sfide Principali dell'Anomaly Detection

Negli ultimi anni, l'integrazione di sistemi intelligenti e dell'Internet of Things (IoT) all'interno dei processi industriali ha contribuito a generare una mole di dati senza precedenti. In questo contesto, garantire l'integrità, la sicurezza e l'affidabilità dei dati è un requisito fondamentale per garantire continuità operativa. Il rilevamento delle anomalie, diventa centrale per far fronte a questa necessità, configurandosi come strumento cardine per identificare tempestivamente guasti meccanici, difetti di produzione, o minacce informatiche prima che diventino fermi macchina o rischi per la sicurezza. Nonostante i grandi successi del Deep Learning e del Machine Learning in scenari controllati, trasferire queste tecnologie in realtà operative presenta ostacoli non banali.[20][27][21]

Per riuscire a progettare soluzioni robuste e dispiegabili sul campo, risulta indispensabile analizzare tre sfide principali:

1. Sbilanciamento delle classi.

   I dataset, in particolare quelli industriali, presentano una rarità di dati anomali rispetto a quelli che definiscono il normale funzionamento. In linee di produzione efficienti, infatti, i dati relativi ai guasti sono estremamente sporadici. Questa asimmetria estrema rende approcci classici spesso inefficaci. Essi, infatti, tendono a soffrire di un bias sistematico verso la classe maggioritaria (quella normale), non riuscendo a generalizzare e riconoscere i rari eventi critici.[27] [20]

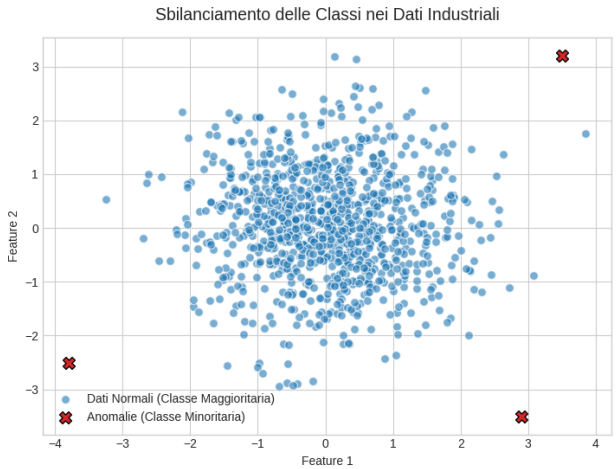


Figura 1.1: Rappresentazione dello sbilanciamento estremo tra dati di normale funzionamento e rari eventi anomali.

2. Variabilità ed eterogeneità dell'anomalia.

Le anomalie presentano caratteristiche di estrema eterogeneità e una natura senza confini definiti. Un malfunzionamento non si manifeste quasi mai secondo un pattern identico, ma anzi può configurarsi come un picco improvviso, un comportamento collettivo anomalo o un'anomalia contestuale. Poiché i guasti sono infinitamente diversificati, risulta impossibile poter mappare tutte le possibili configurazioni di anomalia all'interno di un singolo modello. Per questo, l'algoritmo deve possedere una spiccata capacità di astrazione, riuscendo a catalogare come anomalia qualsiasi entità fuori dallo spazio "normale" anche se mai vista in addestramento.[27] [20][21]

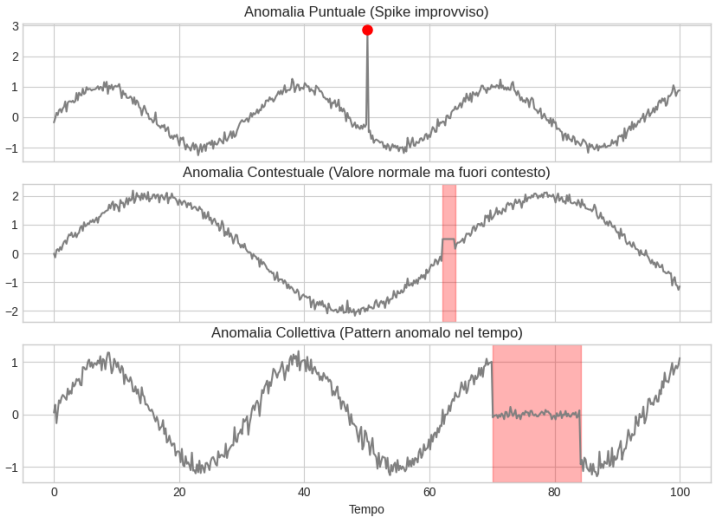


Figura 1.2: Rappresentazione delle tre diverse tipologie di anomalia in una serie temporale: puntuale, contestuale e collettiva.

3. Vincoli e complessità dei contesti industriali.

   I dati sensoriali di una fabbrica sono strettamente legati ad una forte dipendenza del contesto. Un determinato valore di vibrazione o di temperatura può essere considerato perfettamente normale in un determinato momento, come ad esempio, in una particolare stagione, o in una determinata fase produttiva, ma può essere considerata un'anomalia in altri momenti. I confini tra anomalia e normalità, sono sfumati e ambigui. I campioni presentano spesso fluttuazioni insolite, dovute a tolleranze industriali che potrebbero ricalcare i tratti delle anomalie, generando falsi positivi. Il modello deve, quindi, essere progettato con l'integrazione di feature contestuali per evitare che il modello risulti rigido e non adattabile ai cambi di scenario. Diventa fondamentale riuscire ad implementare una fase di pre-elaborazione per mitigare l'effetto dei disturbi sensoristici.[27] [20][21]

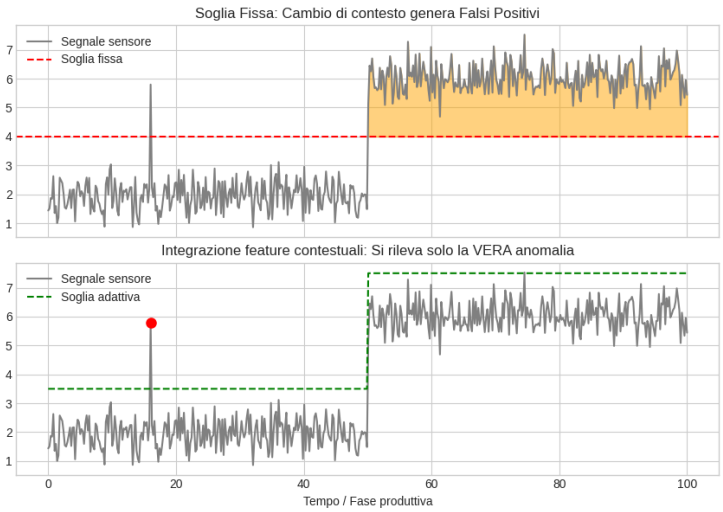


Figura 1.3: Confronto tra l'uso di una soglia di anomalia fissa (che genera falsi positivi al cambio di fase produttiva) e una soglia adattiva basata sul contesto.

1.1.2 Definizioni dei Paradigmi

Le sfide appena descritte — in particolare la rarità dei difetti e l'imprevedibilità del contesto industriale — rendono impossibile l'adozione di un'unica architettura universale. A seconda della disponibilità, della qualità e della mole di dati raccolti, la progettazione del modello deve adattarsi seguendo approcci di addestramento differenti. Vengono, quindi, definiti tre principali paradigmi che hanno guidato lo sviluppo delle metodologie di Visual Anomaly Detection. [26]

- **Paradigma Supervisionato (Supervised Visual Anomaly Detection)**

  Il modello supervisionato (o apprendimento supervisionato) si basa su un paradigma fondamentale: l'algoritmo impara a compiere una determinata associazione o previsione partendo da un dataset di addestramento strutturato in **coppie di input e output attesi (etichette o labels)**. Questo paradigma funziona tramite una guida diretta:

  - **Presenza di etichette (Labels):** ogni dato di input è associato ad una risposta corretta definita da un supervisore umano. Il modello viene quindi addestrato esplicitamente a riconoscere il difetto e a classificarlo come tale.
  - **Definizione dei Confini di Classe:** ricevendo input di entrambe le categorie (buone e difettose), il modello supervisionato riesce a definire i confini decisionali (class boundaries) con estrema precisione, riducendo falsi allarmi e identificando esattamente un oggetto conforme da un non conforme.[2]

Le etichette sono soggette a differenti livelli di granularità in base al diverso metodo applicato. Nella supervisione a livello di immagine (Weakly Supervised) viene fornito solo un flag booleano (ad esempio 0 per sano, 1 per difettoso); mentre, nella supervisione a livello di Pixel (Pixel-level Supervised), viene richiesto uno sforzo umano più minuzioso, in quanto è necessario il disegno di maschere millimetriche attorno ad ogni singolo difetto. Quest'ultimo metodo, è molto potente ma estremamente costoso e lento e spesso impraticabile in contesti complessi. Nonostante l'elevata accuratezza del modello a livello teorico, però, i paradigmi supervisionati presentano ostacoli molto impattanti quando applicati a scenari reali. Essi, difatti, necessitano di grandi quantità di dati che siano esemplificativi per ogni classe; tuttavia, nelle linee di produzione reali, i difetti non sono così numerosi. Il modello finisce per andare in **overfitting**: a causa di dataset di training sbilanciati, con una presenza minima di difetti, il modello memorizza perfettamente i difetti visionati, ma perde la capacità di generalizzare. I difetti sconosciuti, in questo modo, verranno classificati come "sani" solamente a causa di una non conformità con i difetti appresi.[2]

- **Paradigma non Supervisionato (Unsupervised Visual Anomaly Detection)**

  L'approccio non supervisionato consente di costruire modelli di rilevamento senza dover fare affidamento ad etichette o campioni annotati. In questo scenario, infatti, non sono presenti dati che addestrino il modello anticipandogli cosa sia "normale" e cosa sia "difettato". [7] Il paradigma non supervisionato si basa su alcune premesse fondamentali:

  – scarsità statistica delle anomalie: si assume, come visto anche in precedenza, che le anomalie sia rare e minoritarie;
  – differenziazione della norma: l'obiettivo non è raggruppare oggetti simili, ma determinare i campioni che sono più distanti o differenti dalla maggioranza dei dati;
  – deviazione dal pattern: l'identificazione dell'anomalia avviene perché non sufficientemente simile al resto dei dati, e la differenza è misurata tramite specifiche caratteristiche, le **feature**.[12]

  L'approccio non supervisionato presenta alcuni vantaggi, tra cui sicuramente l'abbattimento di tempi e costi. Non sorge, difatti, la necessità di etichettare manualmente e capillarmente i campioni. Un ulteriore vantaggio è il rilevamento delle anomalie sconosciute, in quanto esse vengono riconosciute attraverso l'analisi della differenza tra il campione e il modello di "normalità".[7] Il modello si presenta, inoltre, robusto rispetto alla contaminazione dei dati, riuscendo ad isolare e apprendere la normalità anche in presenza di dataset "sporchi".[15]

- **Paradigma Semi-Supervisionato (Semi-Supervised Anomaly Detection** Il modello semi supervisionato utilizza un set di addestramento che contiene esclusivamente istanze etichettate della classe "normale".

Non è necessario quindi possedere delle etichette per la classe delle anomalie. Questo paradigma, risulta anch'esso essenziale data la nota rarità di anomalie e istanze "non normali". Per far fronte a queste necessità, si utilizzano tecniche a singola classe, che definiscono un confine decisionale, un **boundary**, attorno alla classe negativa, permettendo di isolare la regolarità dei dati normali rispetto gli anomali. Questo metodo, consente di rilevare tipologie di anomalie inedite che non erano mai apparse durante l'addestramento. Gli algoritmi che seguono questo paradigma si suddividono in due categorie principali:

- individuali: operano come modelli singoli. Tra le logiche che caratterizzano questi modelli possiamo distinguere i *modelli lineari*, quelli *basati sulla prossimità*, i *modelli probabilistici*, e le *reti neurali*.
- ensemble: riunisce e fonde più logiche sfruttando i risultati di molteplici classificatori individuali. La regola alla base è che una collezione di classificatori quasi identici non potrà mai superare le prestazioni dei suoi singoli componenti. Esempi possono essere *l'isolation forest* e *l'extreme gradient boosting outlier detection*.

Il paradigma presenta sicuramente vantaggi quali la scoperta di anomalie conosciute (Zero-Day) e l'efficacia in casi di sbilanciamento delle classi. Allo stesso tempo però, il modello è sensibile al "rumore" nei dati e potrebbe presentare un degrado delle prestazioni in contesti specifici, come ad esempio l'elaborazione di dati esclusivamente con feature di tipo nominale.[24]

In sintesi, l'analisi dei paradigmi... [Testo Scelto] In sintesi, l'analisi dei paradigmi di apprendimento evidenzia come la scelta dell'approccio dipenda strettamente dalla natura e dalla disponibilità dei dati industriali. Sebbene i metodi supervisionati offrano un'elevata precisione, la rarità intrinseca delle anomalie spinge la ricerca verso soluzioni semi-supervisionate, capaci di modellare la "normalità" in modo flessibile e resiliente. Tuttavia, la sola definizione del paradigma non è sufficiente a garantire il successo in contesti operativi complessi. Risulta quindi fondamentale analizzare come queste logiche siano state declinate concretamente all'interno della letteratura scientifica recente.

1.2 Il Benchmark MVTec AD e le Metriche di valutazione

Al fine di validare l'efficacia degli algoritmi di *Anomaly Detection* nel contesto industriale, è indispensabile avere un quadro sperimentale standardizzato e condiviso dalla comunità scientifica. Per molto tempo, l'assenza di dati reali e uniformi ha limitato le possibilità di un confronto oggettivo fra le diverse architetture. Questa lacuna è stata colmata dall'introduzione del dataset **MVTec**

Anomaly Detection (MVTech AD), che si è affermata come il benchmark di riferimento per il controllo di qualità industriale.

Verrà analizzata la struttura del dataset, evidenziandone le sfide geometriche poste dalle diverse categorie di materiali contenuti, per poi definire le metriche statistiche utilizzate per misurare e confrontare la sensibilità dei modelli.

1.2.1 Analisi del Benchmark: Texture vs Oggetti Strutturati

Il dataset MVTech AD originale è stato introdotto per far fronte alla carenza di dataset reali e comprensivi per il rilevamento non supervisionato delle anomalie. È composto da 5354 immagini a colori e ad alta risoluzione. Le immagini sono così suddivise:

- **5 categorie di Texture:** rappresentano superfici continue e si suddividono in texture regolari e irregolari. Includono legno, pelle, tappeti, grigle e piastrelle.

- **10 categorie di Oggetti:** rappresentano elementi con confini definiti. Includono oggetti rigidi, come ad esempio dadi di metalli, viti, bottiglie, pillole; oggetti deformabili, come cavi ed elementi naturali.

Il database contiene oltre 70 tipi diversi di difetti. Sono presenti sia difetti superficiali, come graffi, ammaccature, sia difetti strutturali gravi, come parti mancanti o piegate. Inoltre, per poter permettere non solo il rilevamento dell'anomalia ma anche la localizzazione esatta della stessa, il DB fornisce 1888 maschere di *ground truth* che consistono in annotazioni estremamente precise per consentire la localizzazione della posizione del difetto per ogni singolo pixel. [4]

Il dataset ha subito negli anni delle evoluzioni per far fronte le limitazioni che sono sorte dall'avanzamento e dall'affinamento delle tecniche di analisi.

In particolare, è stato rilasciata una prima versione evoluta del dataset, in particolare **MVTech LOCO AD (Logical Constraints Anomaly Detection**. Sviluppato per far fronte alle anomalie logiche, con 3644 immagini con prodotti che sono fisicamente intatti, ma presentano delle violazioni delle regole di assemblaggio, come ad esempio un pezzo valido ma inserito nello scomparto errato.[18]

Negli ultimi anni, gli aggiornamenti del database si sono rifocalizzati sulle anomalie strutturali, estremizzandone la complessità visiva e ottica. **MVTec AD 2**, ha introdotto il caos e i limiti fisici del mondo reale, senza cambiare la logica del difetto. Con l'evoluzione delle tecnologie e dei metodi, i benchmark standard avevano raggiunto un limite fisiologico che non permetteva il confronto delle performance in caso di algoritmi strutturalmente e architetturalmente superiori, per questo si è reso necessario avere un nuovo standard per sbloccare il progresso della ricerca.

In questa versione aggiornata sono presenti non solo oltre 8000 immagini ad alta risoluzione, ma introduce anche nuovi scenari di analisi che replicano i casi più ostici dell'industria moderna. Le innovazioni principali sono:

- **Merci sfuse e oggetti sovrapposti:** spesso elementi come riso o tasselli da muro, sono ammucchiati, in posizioni casuali e sovrapposti, e quindi è richiesta una comprensione semantica molto più profonda.
- **Trasparenza e semi-trasparenza:** materiali come fiale di vetro o gelatine generano riflessi, rifrazioni e distorsioni naturali, che rendono complessa la distinzione tra un difetto interno e un gioco di luce.
- **Alta variabilità "normale":** molti materiali presentano micro-difetti tollerati. L'algoritmo deve quindi imparare ad ignorare il "rumore di fondo" e rilevare solo le anomalie strutturali.
- **Difetti microscopici:** sono stati introdotti difetti microscopici rispetto la risoluzione totale dell'immagine, così da poter testare la capacità dei modelli di estrarre feauture a grana finissima senza perdersi nel contesto.

MVTec AD 2, si propone quindi come benchmark adatto a testare la robustezza percettiva dei modelli. [13]

1.2.2 Criteri di Validazione e Metriche di Performance

Nel campo della visione artificiale, il rilevamento non supervisionato delle anomalie e la loro precisa localizzazione rappresentano sfide cruciali e di grande applicazione pratica. [29] Dalla pubblicazione originale del dataset MVTec AD, la rapida e continua evoluzione degli algoritmi ha spinto le prestazioni dei modelli verso una quasi totale saturazione. [29] Di conseguenza, per valutare oggettivamente i reali progressi dei nuovi algoritmi, è stato necessario l'impiego di metriche di valutazione rigorose, capaci di quantificare accuratamente sia l'identificazione globale dell'anomalia sia la sua esatta localizzazione spaziale. [29]

- **Image-level AUROC**

  Questa metrica rappresenta lo standard per valutare le prestazioni di *rilevamento* globale. Essa quantifica la capacità del modello di assegnare all'intera immagine un punteggio complessivo, permettendo di discriminare in modo affidabile un campione sano da uno difettoso. [29] Nel contesto di ispezione industriale, questa misurazione è fondamentale per determinare il livello di affidabilità dei sistemi automatizzati di scarto. Le valutazioni comparative basate sull'Image-level AUROC hanno ampiamente dimostrato che le moderne architetture basate sulla rappresentazione (representation-based) tendono a ottenere risultati nettamente superiori rispetto ai classici approcci basati sulla ricostruzione dell'immagine (reconstruction-based). [29]

- **Pixel-level AUROC**

  I sistemi di visione artificiale, oltre a identificare un pezzo difettoso, devono essere in grado di localizzare l'esatta posizione dell'anomalia. L'AUROC a livello di pixel valuta tale accuratezza spaziale trattando ogni singolo pixel dell'immagine in modo indipendente. [29] La metrica calcola l'area sotto la curva ROC basandosi sulla correttezza della previsione (sano/anomalo) fornita dal modello per ciascun pixel. [29]

  Tuttavia, studi recenti hanno fatto emergere una forte criticità statistica legata a questa misurazione: raggruppando tutti i pixel dell'immagine in modo indiscriminato, la metrica subisce un forte sbilanciamento di classe. [5] Difatti, modelli poco precisi nella localizzazione del difetto possono comunque ottenere punteggi complessivi artificialmente elevati per il solo fatto di classificare correttamente le vaste, e numericamente preponderanti, aree di sfondo sano, fallendo però nell'identificazione della vera e propria anomalia. [5]

- **PRO (Per-Region Overlap) e AU-PRO**

  Per superare i limiti statistici dell'AUROC a livello di pixel, è stata introdotta la metrica PRO, particolarmente efficace quando il dataset è dominato da anomalie di piccole dimensioni. [29]

  Il PRO valuta le prestazioni di localizzazione misurando l'esatta percentuale di sovrapposizione tra le previsioni binarie dell'algoritmo e le singole regioni del *ground truth* (la maschera reale del difetto). [29] A differenza del raggruppamento indiscriminato dei pixel, il PRO calcola questa sovrapposizione in modo indipendente per ciascuna componente connessa dell'anomalia; ciò garantisce che difetti di dimensioni molto diverse (come un micro-graffio o una vasta ammaccatura) abbiano lo stesso peso matematico, impedendo alle anomalie più estese di dominare la valutazione finale. [29] La misura definitiva, denominata AU-PRO, si ottiene calcolando l'area sotto la curva dei valori PRO in funzione dei tassi di falsi positivi tollerabili (FPR). [29]

  L'adozione di questa metrica ha rivelato che, a fronte di scenari industriali reali e altamente complessi (come quelli introdotti nel dataset MVTec AD 2), le prestazioni medie dei modelli all'avanguardia risultano ancora fortemente deficitarie, crollando al di sotto del 60% in termini di AU-PRO [14].

1.3 Obiettivi e Struttura della Tesi

Il presente lavoro di tesi si propone l'obiettivo di analizzare in modo critico e comparativo le prestazioni delle principali architetture di Deep Learning nello scenario del rilevamento non supervisionato delle anomalie industriali (Visual Anomaly Detection). In particolare, l'indagine mira a valutare il comportamento di modelli basati su paradigmi teorici differenti — quali il modellamento geo-

metrico degli embedding, i Normalizing Flows e la distillazione della conoscenza — evidenziandone i punti di forza e le limitazioni intrinseche quando applicati alle due macro-categorie del dataset MVTec AD: le texture e gli oggetti strutturati. Un ulteriore obiettivo cardine consiste nel verificare l'impatto di strategie mirate di fine-tuning e nell'investigare la sensibilità delle reti a variazioni delle caratteristiche dell'immagine, come il colore e l'illuminazione.

Al fine di rispondere a tali obiettivi, l'elaborato è strutturato secondo il percorso qui delineato:

- Il **Capitolo 2** offre una rassegna approfondita dello stato dell'arte e della tassonomia dei metodi, tracciando l'evoluzione storica dai modelli generativi classici fino alle più moderne architetture basate su embedding (PaDiM), Normalizing Flows (FastFlow) e Teacher-Student (RD4AD).
- Il **Capitolo 3** descrive la metodologia e la configurazione sperimentale adottata, illustrando i criteri di selezione dei modelli oggetto dello studio, i framework software impiegati e le strategie di riaddestramento parziale.
- Il **Capitolo 4** è dedicato alla presentazione e alla discussione dei risultati quantitativi, analizzando i dati prestazionali sia a livello globale sia attraverso una scomposizione verticale tra i subset "Oggetti" e "Texture".
- Il **Capitolo 5** rappresenta il nucleo critico e qualitativo del lavoro; in esso vengono esaminate le mappe di calore (anomaly maps), l'impatto della conversione in scala di grigi (Grayscale) e viene condotta un'accurata analisi dei casi di fallimento (*failure analysis*).
- Il **Capitolo 6**, infine, trae le conclusioni del lavoro svolto, proponendo linee guida operative per il contesto industriale, evidenziando i limiti della ricerca e delineando le prospettive e gli sviluppi futuri.

Capitolo 2

Stato dell'Arte

La visual anomaly detection rappresenta uno dei pilastri fondamentali e maggiormente sfidanti dell'apprendimento automatico e della computer vision.

A differenza dei sistemi di apprendimento tradizionali, che operano in contesti statici e chiusi, i modelli di AD nascono per gestire l'incertezza intrinseca degli ambienti dinamici e aperti, con l'obiettivo di riuscire ad estrarre, caratterizzare e modelare i pattern unicamente dai dati normali disponibili, sfruttando questa conoscenza pregressa per identificare gli elementi sconosciuti, ovvero le anomalie.[26][7]

Questo paradigma trova ampie applicazioni nel mondo reale, suddivise principalmente in tre grandi macro-aree[26]:

1. **Intelligent Manufacturing:** ispezione delle superfici e controllo qualità automatizzato per identificare i difetti di produzione.
2. **Medical Image Analysis:** supporto alle diagnosi mediante l'individuazione automatica di lesioni, masse tumorali o anomalie dei tessuti.
3. **Intelligent Security e Video Surveillance:** rilevamento di eventi rari, incidenti o comportamenti sospetti nei flussi stradali o nella sorveglianza pubblica.

2.0.1 Classificazione per Granularità

Come citato nel capitolo 1, la letteratura scientifica suddivide i metodi visivi in due categorie principali, in base alla risoluzione spaziale e alla granularità richiesta dall'output [26]:

- **Image-level Detection:** il modello esprime un giudizio binario o un punteggio (*anomaly score*) riferito all'intera immagine, indicando se contenga o meno un'anomalia.
- **Pixel-level Detection:** il modello esegue una segmentazione dell'immagine assegnando a ciascun pixel un punteggio di anomalia così da generare una mappa di calore (*anomaly map*).

Tabella 2.1: Tassonomia dei metodi di Visual Anomaly Detection (AD) per dominio e paradigma di apprendimento.

Dominio	Paradigma di Apprendimento	Architetture e Approcci Principali
Image Domain	Unsupervised Learning	CAEs, GANs, Student-Teacher, Normalizing Flows
	Self / Semi-Supervised Learning	SVDD, Approcci basati su feature geometriche
Video Domain	Unsupervised Learning	Spatio-Temporal CAEs, GANs, Modelli Ibridi (CNN-
	Weakly Supervised Learning	Multiple Instance Learning, CNN pre-addestrate

L'evoluzione tecnologica ha vissuto una netta divisione con l'avvento del Deep Learning. I metodi si sono evoluti attraverso tre principali approcci all'addestramento [7]:

- Supervised Learning: richiede un dataset bilanciato ed etichettato sia con campioni normali che con tutte le possibili classi di difetti.
- Unsupervised Learning: rappresenta lo standard *de facto* della disciplina, addestrando il modello esponendolo solo a campioni privi di difetti.
- Weakly Supervised Learning: utilizza etichette a livello di video o gruppo di immagini, senza specificare in quale frame o pixel fosse l'anomalia.

2.1 L'era del Pre-Deep Learning

Prima della diffusione delle reti neurali, la computer vision si affidava ad una pipeline sequenziale a due stadi: l'estrazione di feature artigianali (*handcrafted features*) seguita da un classificatore o stimatore statistico superficiale.[26]

I vettori più utilizzati si basavano su variazioni geometriche, fotometriche e di texture locali:

- **Descrittori di apparenza e istogrammi**: lo *Scale-Invariant Feature Transform (SIFT)* per l'estrazione di punti di interessi invarianti alla scala, gli *Histograms of Oriented Gradients (HOG)* per catturare le informazioni strutturali e i contorni visivi e i *Local Binary Patterns (LBP* per la codifica locale delle trame di texture superficiali.[26][8]
- **Metodi Basati su Filtri e Trasformate:** l'applicazione di banchi di filtri energetici come gli operatori di Canny e Sobel, i filtri di Gabor e le trasformate sequenziali di Fourier e Waveket per l'analisi energetica delle risposte spaziali e isolare le discontinuità.[23]

Una volta estratti i descrittori locali, i meccanismi decisionali si dividevano in tre principali filoni:[26]

1. **Stima di densità parametrica e non parametrica**

   La modellazione della distribuzione dei dati avviene tramite modelli misti gaussiani (GMM) o stimatori di Kernel della densità (KDE). Se la probabilità logaritmica di un campione di test risulta inferiore ad una certa soglia, viene classificato come anomalo.

2. **One-Class Classification (OCSVM e SVDD)**

   Utilizza tecniche geometriche volte a mappare le feature in uno spazio a dimensionalità superiore per definire un confine decisionale ottimale attorno alla sola classe normale.

3. **Modello temporali e di contesto**

   Vengono utilizzati i Campi Aleatori di Markov (MRF) per combinare i descrittori visivi con le variazioni dinamiche calcolate tramite flussi ottici locali (HOF).

Questi approcci, però, soffrivano pesantemente della *"Curse of dimensionality"* quando applicati direttamente a immagini ad alta risoluzione. Inoltre, la loro natura non end-to-end impediva l'ottimizzazione congiunta delle feature e del criterio di rilevamento, limitando la robustezza algoritmica difronte a rumore di fondo, variazioni di illuminazione e ombre complesse.[26]

2.2 L'Evoluzione dei metodi basati sul Deep Learning

L'integrazione del Deep Learning ha consentito di superae i limiti strutturali delle feature manuali, consentendo l'estrazione automatica di rappresentazioni semantiche multilivello. Inoltre, con il successo delle CNN, l'attenzione si concentrata su come sfruttare la loro capacità di astrazione per sviluppare approcci end-to-end. I metodi moderni di AD non supervisionata si articolano in cinque principali famiglie.[7]

2.2.1 Metodi basati sulla Ricostruzione (Reconstruction-based)

Il concetto principale si fonda sul concetto di compressione ridondante della conoscenza. Il modello viene addestrato a mappare un'immagine normale in uno spazio a bassa dimensionalità per poi tentare di ricostruire l'input originale tramite un percorso inverso.[26]

L'assunto alla base del modello è che questo non essendo mai stato esposto alle anomalie durante il training, non sarà in possesso di coefficienti necessari per descriverle, generando un errore di ricostruzione elevato quando sono presenti dei difetti.

- **Formule Chiave** Nella pipeline basate su autoencoder e misure strutturali come lo *Structural Similarity Index (SSIM)*, l'anomaly score spaziale, viene misurato combinando la distanza euclidea dei pixel e la consistenza strutturale locale:[8][26]

$$\mathcal{L}_{\text{rec}}(x, \hat{x}) = (1 - \alpha) \cdot \|x - \hat{x}\|_2 + \alpha \cdot (1 - \text{SSIM}(x, \hat{x}))$$

Nei modelli generativi avanzati come ad esempio **AnoGAN**, la ricerca dell'immagine normale più vicina richiede un'ottimizzazione iterativa nello spazio latente z. La funzione di perdita per aggiornare il vettore latente unisce la perdita di embedding residua e la discriminazione delle feature estratte da un livello intermedio del discriminatore (D_λ):[26]

$$\mathcal{L}_{\text{AnoGAN}}(z_\gamma) = (1-\lambda) \cdot \|x - G(z_\gamma)\|_1 + \lambda \cdot \|D_\lambda(x) - D_\lambda(G(z_\gamma))\|_1$$

- **Evoluzioni e varianti** Nel tempo i metodi sono stati soggetti a studi che hanno portato alla loro evoluzione e alla definizione di varianti degli stessi. In particolare:
  - **CAVGA (Constrained Adversarial Variational General Architecture)**
    Introduce meccanismi di attenzione visiva integrati tramite mappe Grad-CAM per costringere la rete a coprire l'intera area normale dell'immagine, bloccando contemporaneamente le risposte sulle aree esterne.
  - **DFR (Deep Feature Reconstruction)**
    Sposta il processo di ricostruzione dallo spazio dei pixel a quello delle feature multilivello estratte da CNN pre-addestrate, riducendo i falsi positivi su bordi complessi.

  Bisogna sottolineare che sebbene i modelli puramente ricostruttivi siano intuitivi e precisi nella localizzazione spaziale, possono generalizzare in modo eccessivo, riuscendo a ricostruire parzialmente anche le anomalie visive, invalidando l'ipotesi di partenza.[7]

2.2.2 Metodi basati sui flussi di normalizzazione (Normalizing Flow-Based

I Normalizing Flow sono architetture potenti capaci di apprendere trasformazioni matematiche biettive e invertibili tra la complessa distribuzione visiva delle immagini e una funzione di densità di probabilità semplice e nota.

Durante la fase di training, il modello punta a massimizzare la verosomiglianza logaritmica dei campioni normali. Nella fase di test, invece, l'immagine viene elaborata dalla rete e la densità di probabilità risultante funge direttamente da punteggio di anomalia (più bassa è la probabilità calcolata, più è alta la probabilità che si tratti di un difetto).[7]

- **Fomule Chiave** Alla base del metodo c'è il teorema del cambiamento di variabili. Se $Z = f(x)$ è la mappatura verso lo spazio latente Gaussiano, la densità di probabilità di un campione x si calcola considerando il determinante della matrice Jacobiana (J_f) della trasformazione:

$$p(x) = p(f(x)) \cdot |\det J_f(x)|$$

In fase di training, la rete massimizza la log-verosomiglianza sui dati normali:

$$\text{Log}(p(x)) = \text{Log}(p(f(x))) + \text{Log}(|\det J_f(x)|)$$

- **Evoluzioni e varianti** Nel tempo, lo studio del metodo ha portato evoluzioni dello stesso e lo sviluppo di varianti, come:
  - **DifferNet** Considerato uno dei primi modelli ad applicare i NF all'ispezione visiva, estraendo delle feature multi-scala per la classificazione a livello di immagini.
  - **CFlow-AD (Conditional Flow)** Il modello introduce blocchi di flussi condizionali per elaborare le piramidi di feature, abilitando la localizzazione del difetto a livello di singolo pixel.
  - **FastFlow** Questo modello risolve un limite enorme rispetto i precedenti che, per poter processare i dati, dovevano appiattire la matrice dell'immagine in un vettore 1D, distruggendo le relazioni spaziali. FastFlow utilizza reti convoluzionali 2D per i flussi, preservando le vicinanze strutturali dei pixel e migliorando la precisione.[7]

2.2.3 Metodi basati sulla Rappresentazione (Rapresentation-based

In questa categoria metodologica viene esclusa la complessa fase di addestramento deep del modello. Vengono impiegate reti neurali pre-addestrate esclusivamente come estrattori potenti di descrittori visivi *(feature extractor)*. L'anomaly score si ricava calcolando le distanze geometriche statistiche tra il vettore di test e l'insieme di vettori normali presenti in memoria.[7]

- **Formule Chiave** Gli algoritmi appartenenti a questa categoria superano la semplice distanza euclidea, utilizzando delle metriche statistiche che considerando la varianza. Per un patch (una specifica regione dell'immagine), l'anomalia viene calcolata utilizzando la distanza di Mahalanobis:

$$D_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

  Dove x è la feature in esame, μ è il vettore medio calcolato sulle immagini normali in quella specifica posizione nello spazio, e Σ^{-1} è la matrice di covarianza inversa.

- **Evoluzioni e Varianti** Gli studi, dopo la definizione di questo metodo, hanno portato a diversi sviluppi, che hanno definito più varianti, tra le principali troviamo:
  - **SPADE** Estrae piramidi di feature e utilizza un classificatore KNN per cercare i vettori normali più simili per ogni singolo pixel. Risulta molto preciso ma esecutivamente lento data la ricerca dettagliata.[26]

- **PaDiM (Patch Distribution Modeling)** Modella la distribuzione delle patch tramite Gaussiane Multivariate per calcolare la distanza in modo efficiente senza necessariamente richidere una ricerca KNN continua.[7]
- **PatchCore** Attualmente, viene considerato lo stato dell'arte in ambito industriale. Raccoglie le feature in una *Memory Bank*, ma comprimendo l'archivio così da poter mantenere i FPS elevati senza saturare la RAM. Per far ciò utilizza un algoritmo che salva solo le feature essenziali, ovvero il *greedy coreset subsampling*.
- **CFA (Coupled-hypersphere-based Feature Adaptation** Supera il problema del "bias" delle reti pre-addestrate usando il transfer learning per adattare le feature estratte al dominio specifico, proiettandole su ipersfere matematiche.

2.2.4 Metodi basati sulla Data Augmentation

I metodi non supervisionati sono caratterizzati da un training con assenza totale di immagini anomali, per questo, questi algoritmi puntano a "fabbricare" difetti sintesici manipolando le immagini normali. Questo fa si che l'Anomaly Detection si trasformi in un compito *self-supervised* di segmentazione: il modello viene quindi allenato a trovare le false anomalie inserite dal programmatore.[7]

- **Formule Chiave** I modelli appena definiti utilizzando le funzioni di perdita classiche della segmentazione supervisionata per separare i pixel veri da quelli alterati.

$$\text{Loss}_{\text{Segmentazione}} = -\left[y \cdot \log(p) + (1-y) \cdot \log(1-p)\right]$$

  Dove

$$y$$

  è la maschera binaria del difetto finto (1 se è presente l'anomalia, 0 se l'immagine è sana) e

$$p$$

  è la previsione della rete.

- **Evoluzione e Varianti** Anche questo metodo presenta più evoluzioni dello stesso.
  - **CutPaste** La logica ritaglia in modo casuale piccole zone dell'immagine sana *(cropping)*, applica variazioni di colore o rotazioni, e le incolla in un altro punto dell'immagine. Un classificatore viene poi addestrato a riconscere se l'immagine è autentica o "incollata".[7]
  - **DREAM** Genera maschere di rumore da sovrapporre come texture. Vengono addestrate due reti simultaneamente: una per ricostruire l'immagine pulita e un discriminatore per tracciare i bordi del difetto simulato.

– **NSA (Natural Synthetic Anomalies** Questo metodo taglia patch da immagini differenti e le fonde in campioni sani utilizzando algoritmi di *Poisson image editing*, simulando irregolarità strutturali con sfumature decisamente realistiche.

Di seguito viene presentata un'analisi comparativa dei macro-approcci descritti, evidenziandone vantaggi e svantaggi sulla base di tre criteri principali: tempo di inferenza, consumo di memoria e accuratezza della localizzazione.

Tabella 2.2: Confronto riepilogativo delle principali famiglie di metodi per la Visual Anomaly Detection.

Categoria	**Modelli Esempio**	**Concetto Base**	**Vantaggi**	**Svantaggi**
Pre-Deep Learning	SIFT, HOG, LBP, GMM, OCSVM	Estrazione feature manuali + modelli statistici/geometrici.	Leggeri e interpretabili.	*Curse of dimensionality*; non end-to-end.
Ricostruzione	AE, VAE, AnoGAN, CAVGA, DFR	Compressione e ricostruzione. L'errore segna l'anomalia.	Intuitivi; buona localizzazione spaziale.	Generalizzazione eccessiva sui difetti.
Normalizing Flow	DifferNet, CFlow-AD, FastFlow	Mappatura biettiva su curva Gaussiana; stima di probabilità.	Base matematica stabile; inferenza veloce.	Addestramento esoso in termini di VRAM.
Rappresentazione	SPADE, PaDiM, PatchCore, CFA	Estrazione feature da reti preaddestrate e confronto distanze.	Altissime prestazioni; nessun training profondo.	Dipendenza dal dominio originario (ImageNet); consumo memoria.
Data Augmentation	CutPaste, DRAEM, NSA	Sintesi di difetti artificiali e task *self-supervised*.	Facile implementazione con loss classiche.	Difficoltà a replicare la fisica dei difetti reali.

2.3 Analisi critica e sfide aperte nell'Anomaly Detection

Nonostante l'introduzione delle architetture del Deep Learning abbiamo contribuito a rivoluzionare le prestazioni dei sistemi della Visual Anomaly Detection, il passaggio dagli ambienti di laboratorio ai contesti applicativi reali presenta ancora degli ostacoli significativi. La letteratura più recente, dopo un'analisi critica ha evidenziato come l'AD in ambito industriale sia frenata da problematiche che vanno oltre la pura precisione algoritmica, coinvolgendo l'acquisizione dei dati, la robustezza ambientale e l'integrazione hardware.[1][16]

Di seguito vengono analizzate le principali sfide ancora aperte che guidano la scena attuale della ricerca scientifica.

2.3.1 Il "Bias" di Dominio e l'Adattamento delle Feature

Attualmente, i metodi considerati allo stato dell'arte per l'applicazione industriale, come ad esempio quelli basati sulla Rappresentazione, si affidano principalmente a reti neutrali pre-addestrate su dataset generici di grandi dimensioni, come ImageNet.[22] Nonostante questa strategia risolva il problema del **cold-start**, ovvero riesca ad addestrare una rete profonda solo con campioni normali, introduce un forte "bias di dominio".[16] Le *feature* estratte dalle immagini naturali, spesso non possiedono la granularità necessaria per riuscire a descrivere in modo ottimale le micro-strutture di una texture industriale o le imperfezioni di un macchinario ad altra precisione; questa dipendenza rende necessario sviluppare nuove tecniche di *transfer learning* e di adattamento al dominio non supervisionato, capaci di sintonizzare i descrittori sulle specifiche peculiarità del task senza invalidare le prestazioni del modello.[16]

2.3.2 Scalabilità Computazionale ed Edge Computing

L'integrazione dei sistemi di rilevamento delle anomalie nelle linee di produzione reali impone spesso l'adozione di paradigmi di *Edge Computing*. In questi casi, gli algoritmi devono essere eseguiti direttamente su dispositivi periferici, quali telecamere smart o nodi IoT, strutturalmente penalizzati da stringenti vincoli in termini di memoria e capacità di calcolo. [17] Il contrasto tra la necessità operativa e i requisiti hardware delle architetture visive, anche le più performanti, genera colli di bottiglia che non si adattano alla produzione in tempo reale. Ad esempio, i modelli basati sulla memorizzazione delle caratteristiche, come PatchCore, sebbene utilizzano strategie di compressione, richiedono comunque l'archiviazione e la consultazione di enormi *Memory Bank* di campioni nominali per riuscire a calcolare i punteggi dell'anomalia in fase di test.[22] Anche l'implementazione del Deep Learning in contesti IoT contribuisce ad introdurre severe criticità legate principalmente all'efficienza energetica e alla latenza decisionale; di conseguenza, l'ottimizzazione strutturale dei modelli e lo sviluppo di nuove soluzioni *Lightweight Deep Learning* rimangono cruciali per colmare il divario tra efficacia teorica e applicabilità industriale.[17]

2.3.3 Ambienti Dinamici e Continual Anomaly Detection

Generalmente, il paradigma della VAD presuppone un ambiente statico, ovvero, il modello apprende una singola definizione di "normalità" che rimane invariata per tutta la durata dell'inferenza. Tuttavia, nel mondo reale i dati sono soggetti a *Concept Drift*: cambiano le condizioni di illuminazione, l'usura dell'hardware, o anche variazioni concettuali del desing del prodotto. Di fronte a questa realtà, i modelli statici perdono calibrazione, generando un numero terribilmente elevato di falsi positivi.[11] La letteratura, per far fronte a queste criticità sta convergendo verso la *Continual Anomaly Detection*, con l'obiettivo di sviluppare dei sistemi in grado di aggiornare in modo incrementale la propria conoscenza della normalità in flussi dati continui, mitigando così la perdita drastica delle

informazioni acquisite in precedenza *(catastrophic forgetting)* durante le fasi di ri-addestramento.[11]

2.3.4 Prospettiva Zero-Shot e i Foundation Models

L'acquisizione di ampi set di dati normali risulta impraticabile in scenari di produzione altamente personalizzati o per piccoli lotti, per questo la ricerca si sta muovendo verso un approccio tendente alla *Zero-Shot Anomaly Detection*. Gli sviluppi più recenti, in particolare, esplorano l'utilizzo di modelli multi-modali molto potenti definiti *Foundation Models*, per rilevare le anomalie senza alcun addestramento specifico sul dominio.[25] Architetture emergenti, come il framework DCS, dimostrano la possibilità di integrare modelli visivo-linguistici per identificare e segmentare difetti complessi guidando l'attenzione della rete attraverso prompt testuali, riducendo così in modo drastico, i costi di setup e introducendo ispezioni industriali generalizzabili.[25]

Capitolo 3

Modelli selezionati e implementazione sperimentale

Il presente capitolo descrive i quattro modelli selezionati per il progetto — **CCW-SSIM**, **PaDiM**, **RD4AD** e **FastFlow** — distinguendo, per ciascuno di essi, i principi metodologici dall'effettiva implementazione adottata. Le descrizioni implementative fanno quindi riferimento al codice, alle configurazioni e agli artefatti presenti nel repository del progetto.

Una distinzione preliminare riguarda il ruolo dei dataset. Nel progetto, **MVTec AD** costituisce il dataset sul quale sono eseguiti l'adattamento al dominio industriale e l'inferenza. **ImageNet**, invece, interviene esclusivamente come sorgente dei pesi pre-addestrati dei backbone utilizzati da PaDiM, RD4AD e FastFlow; non sostituisce MVTec AD come dataset sperimentale. CCW-SSIM non utilizza pesi ImageNet ed è addestrato da zero sui campioni normali di MVTec AD.

3.1 Criteri di selezione e ruolo dei modelli

3.1.1 Inquadramento nei paradigmi metodologici

La selezione copre quattro famiglie metodologiche discusse nel Capitolo 2:

- **CCW-SSIM** [3] appartiene ai metodi basati sulla ricostruzione. Un autoencoder apprende a riprodurre i campioni normali e la discrepanza tra immagine originale e ricostruita è valutata considerando congiuntamente struttura e colore;
- **PaDiM** [9] è un metodo basato sulla rappresentazione. Le feature estratte da una rete pre-addestrata sono modellate, per ciascuna posizione spaziale, mediante una distribuzione gaussiana multivariata;

- **RD4AD** [10] adotta la *Reverse Distillation*. Un encoder *Teacher* congelato fornisce le rappresentazioni di riferimento, mentre un collo di bottiglia e un decoder *Student* apprendono a ricostruirle sui soli campioni normali;
- **FastFlow** [28] impiega flussi di normalizzazione bidimensionali. Trasformazioni invertibili apprendono la distribuzione delle feature normali preservandone l'organizzazione spaziale.

I quattro approcci rappresentano pertanto modalità differenti di costruire un modello della normalità: ricostruzione dell'immagine, stima statistica locale delle feature, ricostruzione distillata delle feature e stima esplicita della loro densità.

3.1.2 Ruolo dei modelli nel progetto

L'implementazione adotta uno scenario *one-class*, comunemente ricondotto alla Visual Anomaly Detection non supervisionata: la fase di adattamento utilizza esclusivamente immagini prive di difetti, mentre le immagini normali e anomale della partizione di test sono impiegate soltanto nella successiva fase di inferenza.

Il termine *addestramento* assume un significato diverso nei quattro casi. Per CCW-SSIM, RD4AD e FastFlow indica l'ottimizzazione di parametri neurali. Per PaDiM indica invece la stima delle statistiche gaussiane sulle feature normali, senza aggiornare il backbone. Tale differenza viene esplicitata nelle sezioni dedicate, evitando di attribuire a tutti i modelli la medesima procedura di ottimizzazione.

3.2 Ambiente e protocollo implementativo comune

3.2.1 Organizzazione del repository e pipeline di esecuzione

Il repository è organizzato in quattro directory principali, denominate `CCW-SSIM`, `PaDiM`, `RD4AD` e `FastFlow`. Ogni directory contiene un punto di ingresso `Main.py`, una configurazione dedicata e i moduli per caricamento dei dati, costruzione del modello, adattamento e inferenza. Il notebook `pipeline_tesi.ipynb` coordina l'esecuzione delle pipeline mantenendo separati gli ambienti dei modelli.

Il dataset è conservato nella directory centrale `MVTec_Data`. Le directory locali dei modelli vi accedono mediante collegamenti simbolici, così da evitare copie divergenti dei dati. L'esecuzione comune è articolata nelle seguenti fasi:

- adattamento del modello sui campioni normali presenti in `train/good`;
- caricamento del checkpoint o delle statistiche apprese;
- inferenza sulle immagini della partizione di test;

- salvataggio delle mappe di anomalia e dei metadati di esecuzione.

Il modulo `experiment_protocol.py` registra manifesti e impronte dei file impiegati, rendendo esplicite la configurazione e la provenienza degli artefatti. Questa organizzazione separa il codice specifico dei modelli dal protocollo condiviso.

3.2.2 Ambiente software e hardware

Gli ambienti registrati nei manifesti del progetto utilizzano Linux su architettura `x86_64` e Python 3.11.15. CCW-SSIM è implementato con TensorFlow 2.15 e Keras 2.15, mentre PaDiM, RD4AD e FastFlow utilizzano PyTorch 2.13.0 e Torchvision 0.28. Le configurazioni rilevate sono riassunte nella Tabella 3.1.

Modello	**Framework**	**Supporto GPU registrato**
CCW-SSIM	TensorFlow 2.15, Keras 2.15	CUDA 12.2, cuDNN 8
PaDiM	PyTorch 2.13.0, Torchvision 0.28	CUDA 13.0
RD4AD	PyTorch 2.13.0, Torchvision 0.28	CUDA 13.0
FastFlow	PyTorch 2.13.0, Torchvision 0.28	CUDA 13.0

Tabella 3.1: Ambienti software registrati per le implementazioni.

Le dipendenze condivise comprendono NumPy 1.26.4, OpenCV 4.11.0, Pillow 12.3, SciPy 1.17.1, scikit-image 0.26 e scikit-learn 1.9. La gestione dell'ambiente radice è definita dai file `pyproject.toml` e `uv.lock`. I manifesti indicano l'impiego di una GPU NVIDIA GeForce RTX 4070 Ti.

3.2.3 Dataset MVTec AD e categoria leather

MVTec AD [4] raccoglie immagini industriali ad alta risoluzione, organizzate in categorie di oggetti e texture. Il repository conserva le quindici categorie del dataset; le configurazioni e i manifesti verificabili per i quattro modelli fanno tuttavia riferimento alla categoria `leather`.

Per tale categoria, la partizione `train/good` contiene 245 immagini normali. La partizione di test contiene 124 immagini: 32 normali e 92 appartenenti alle tipologie di difetto `color`, `cut`, `fold`, `glue` e `poke`. Per le immagini difettose sono disponibili 92 maschere nella directory `ground_truth`. La medesima organizzazione dei dati è condivisa da tutte le implementazioni.

3.2.4 Gestione di configurazioni, checkpoint e artefatti

Tutte le pipeline fissano il seme pseudo-casuale a 1024. I parametri sono mantenuti nei rispettivi file di configurazione, mentre i pesi o le statistiche apprese

sono salvati in formati specifici per il framework: file HDF5 per CCW-SSIM, file `pickle` per PaDiM e file PTH per RD4AD e FastFlow.

Le mappe di anomalia sono memorizzate come immagini TIFF in virgola mobile a 32 bit, senza quantizzazione preventiva. A ogni esecuzione sono inoltre associati un manifesto JSON e, per i modelli ottimizzati iterativamente, la cronologia di addestramento in formato CSV. I contenuti di tali artefatti non vengono discussi in questo capitolo, che si limita a documentarne produzione e formato.

3.3 CCW-SSIM

3.3.1 Descrizione del modello

Autoencoder e ricostruzione

CCW-SSIM è un metodo basato sulla ricostruzione [3]. Un autoencoder convoluzionale comprime l'immagine in una rappresentazione latente e la ricostruisce. L'addestramento sui soli campioni normali induce la rete a rappresentare regolarità cromatiche e strutturali proprie del dominio osservato.

Indicando con x l'immagine di ingresso e con g_θ l'autoencoder, la ricostruzione è

$$\widehat{x} = g_\theta(x). \tag{3.1}$$

La discrepanza non è calcolata esclusivamente nello spazio RGB: il modello combina una misura strutturale, derivata dalla CW-SSIM, con una componente cromatica percettiva.

S-CIELAB e CW-SSIM

La trasformazione S-CIELAB converte l'immagine RGB nello spazio XYZ, separa l'informazione in canali opponenti e applica filtri spaziali differenti alle componenti di luminanza e crominanza. Dopo la riconversione in CIELAB si ottengono i canali L^*, a^* e b^*, nei quali la distanza cromatica tiene conto della diversa sensibilità del sistema visivo umano alle frequenze spaziali.

La componente strutturale è calcolata su coefficienti complessi ottenuti mediante una piramide steerable. Per due insiemi locali di coefficienti c_x e c_y, la CW-SSIM può essere espressa come

$$S_{\mathrm{CW}}(c_x, c_y) = \frac{2\left|\sum_i c_{x,i} c_{y,i}^*\right| + K}{\sum_i |c_{x,i}|^2 + \sum_i |c_{y,i}|^2 + K}, \tag{3.2}$$

dove $c_{y,i}^*$ è il complesso coniugato e K è una costante di stabilizzazione. La combinazione con la compatibilità cromatica consente di considerare sia le variazioni di struttura sia quelle di colore.

3.3.2 Implementazione nel progetto

Organizzazione del codice e architettura

L'architettura è definita in `NNModels.py`. Addestramento e inferenza sono gestiti rispettivamente da `Training.py` e `Prediction.py`; il preprocessing percettivo è implementato in `ColorUtils.py`, `SpatialFilters.py` e nella directory `Steerables`. La configurazione seleziona la variante seguente:

`Model_noise_skip_bigger_old`.

L'encoder utilizza convoluzioni 4×4 con canali 32, 64, 128, 256 e 2048 e stride rispettivamente 2, 2, 2, 4 e 4. Le attivazioni sono di tipo LeakyReLU. Il decoder è simmetrico e utilizza convoluzioni trasposte 4×4 con canali 256, 128, 64, 32 e 3 e stride 4, 4, 2, 2 e 2. L'ultimo strato ha attivazione lineare. Non sono presenti strati di Batch Normalization o Max-Pooling nella variante configurata.

La rete è inizializzata da zero e non impiega un backbone pre-addestrato su ImageNet. Il codice contiene anche componenti opzionali basati su VGG, ma esse non partecipano alla configurazione selezionata, che usa la perdita `color_cwssim_loss`.

Preprocessing e addestramento

Le immagini sorgente della categoria `leather` hanno dimensione 1024×1024 pixel. Il generatore estrae, da ciascuna immagine normale, 64 patch casuali di dimensione 128×128, che formano il batch effettivo. Sono configurate 200 epoche e 245 passi per epoca, corrispondenti al numero di immagini normali disponibili.

Il preprocessing converte RGB in XYZ mediante scikit-image, applica la trasformazione nei tre canali opponenti, filtra ciascun canale con kernel spaziali 5×5, riconverte in CIELAB e normalizza i valori dividendo per 102. L'ottimizzazione usa Adam con tasso di apprendimento iniziale 10^{-3}; uno scheduler moltiplica tale valore per 0,5 ogni 15 epoche. I tensori della pipeline CCW-SSIM sono elaborati in virgola mobile a 64 bit.

La perdita configurata combina la CW-SSIM sul primo canale Lab con l'errore cromatico sui canali a^* e b^*. La piramide complessa usa sei scale e sei orientazioni, mentre la finestra locale ha lato 7. I checkpoint sono salvati ogni dieci epoche.

Inferenza e salvataggio degli artefatti

La modalità attiva nella pipeline esegue l'inferenza *patchwise*. Ogni immagine è suddivisa in patch 256×256 con stride di 16 pixel; le ricostruzioni sovrapposte sono ricomposte mediante media. La suddivisione del carico è controllata dal parametro `BatchSplits`, impostato a 25.

La mappa locale combina la componente cromatica con la CW-SSIM calcolata a più altezze della piramide e su sei orientazioni. Il codice conserva inoltre

l'inversione del primo asse Lab prevista dalla convenzione storica del decoder. Le ricostruzioni, i checkpoint HDF5, la cronologia CSV, il manifesto JSON e le mappe TIFF costituiscono gli artefatti della pipeline. La funzione sperimentale per la predizione dell'intera immagine è presente nel codice, ma non è selezionata dalla configurazione utilizzata.

3.4 PaDiM

3.4.1 Descrizione del modello

Rappresentazione multiscala

PaDiM, acronimo di *Patch Distribution Modeling*, descrive ogni posizione dell'immagine mediante feature estratte a più livelli da una rete convoluzionale pre-addestrata [9]. Le rappresentazioni dei livelli iniziali mantengono maggiore dettaglio spaziale, mentre quelle più profonde codificano informazioni semantiche più astratte. L'allineamento spaziale e la concatenazione producono un vettore $z_{i,j}$ per ogni posizione (i, j).

Il backbone rimane congelato: ImageNet fornisce una inizializzazione generica dell'estrattore, mentre la distribuzione normale specifica del dominio viene stimata sulle immagini MVTec AD.

Modellazione gaussiana locale

Per ogni posizione spaziale, PaDiM stima una gaussiana multivariata

$$z_{i,j} \sim \mathcal{N}(\mu_{i,j}, \Sigma_{i,j}). \tag{3.3}$$

La discrepanza di una feature di test dalla distribuzione normale locale è espressa mediante la distanza di Mahalanobis

$$d_{i,j} = \sqrt{(z_{i,j} - \mu_{i,j})^{\mathrm{T}} \Sigma_{i,j}^{-1} (z_{i,j} - \mu_{i,j})}. \tag{3.4}$$

La modellazione indipendente delle posizioni conserva l'informazione relativa alla collocazione delle strutture nell'immagine.

3.4.2 Implementazione nel progetto

Backbone e allineamento patchwise

Il modello è definito in `NNModels.py`; adattamento e inferenza sono gestiti da `Training.py` e `Prediction.py`. Il backbone configurato è Wide ResNet-50-2 con pesi ImageNet-1K V1 forniti da Torchvision. I parametri del backbone restano congelati per l'intera procedura.

Le immagini sono ridimensionate a 224×224 e normalizzate con media e deviazione standard ImageNet. Le feature provengono da `layer1`, `layer2` e `layer3`, con 256, 512 e 1024 canali. Le mappe a risoluzione inferiore non

sono semplicemente interpolate: la funzione `embedding_concat` usa operazioni `unfold` e `fold` per allineare e concatenare le feature in corrispondenza delle patch della mappa a risoluzione maggiore, pari a 56×56.

Adattamento statistico

La concatenazione completa produce 1792 canali. Per contenere la dimensionalità statistica, l'implementazione seleziona deterministicamente 550 indici di canale mediante `torch.randperm` con seme 1024. Non viene applicata una proiezione casuale: il sottoinsieme è composto direttamente dai canali selezionati e i relativi indici sono salvati insieme alle statistiche.

Le immagini normali sono elaborate in batch da 32, con due processi di caricamento e ordine non mescolato. La procedura esegue un unico passaggio sul training set, calcola media e covarianza per ogni posizione e aggiunge $0{,}01I$ alla covarianza prima dell'inversione. Non sono presenti ottimizzatore o epoche di aggiornamento dei pesi. Le medie, le inverse delle covarianze e gli indici dei canali sono serializzati in un file `pickle`.

Inferenza e salvataggio degli artefatti

In inferenza, il preprocessing e il sottoinsieme di canali coincidono con quelli usati durante l'adattamento. Per ogni posizione viene calcolata la distanza riportata nell'Equazione 3.4. La mappa 56×56 è riportata alla dimensione originale mediante interpolazione bicubica e regolarizzata con un filtro gaussiano di deviazione standard 4.

La pipeline salva le statistiche in formato `pickle`, il manifesto JSON e una mappa TIFF in virgola mobile a 32 bit per ciascuna immagine di test.

3.5 RD4AD

3.5.1 Descrizione del modello

Reverse Distillation

RD4AD applica la distillazione della conoscenza in direzione inversa rispetto alla configurazione encoder–encoder tradizionale [10]. Il *Teacher Encoder*, preaddestrato e congelato, produce rappresentazioni multiscala. Un decoder *Student* riceve una rappresentazione compressa e tenta di ricostruire le feature normali del Teacher procedendo dalle rappresentazioni profonde verso quelle meno profonde.

Durante l'adattamento, la somiglianza tra le rappresentazioni del Teacher e quelle ricostruite dallo Student guida l'ottimizzazione. In inferenza, la discrepanza locale tra le coppie di feature corrispondenti forma la mappa di anomalia.

Teacher, OCBE e Student Decoder

Tra Teacher e Student è inserito un *One-Class Bottleneck Embedding* (OCBE). Il modulo aggrega le feature provenienti da livelli differenti e restringe il passaggio dell'informazione. L'asimmetria tra encoder congelato, collo di bottiglia addestrabile e decoder riduce la possibilità di trasferire direttamente allo Student tutte le attivazioni del Teacher.

Indicando con $f_k(x)$ la feature del Teacher al livello k e con $\widehat{f}_k(x)$ quella ricostruita, la discrepanza coseno locale impiegata per la localizzazione è

$$a_k(h,w) = 1 - \frac{f_k(h,w)^{\mathrm{T}} \widehat{f}_k(h,w)}{\|f_k(h,w)\|_2 \|\widehat{f}_k(h,w)\|_2}. \tag{3.5}$$

3.5.2 Implementazione nel progetto

Organizzazione dei componenti

L'architettura è definita nel file `NNModels.py`. L'adattamento è gestito da `Training.py` e l'inferenza da `Prediction.py`.

Il Teacher è Wide ResNet-50-2 con pesi ImageNet-1K V1 ed espone le feature di `layer1`, `layer2` e `layer3`. Le immagini sono ridimensionate a 256×256 e normalizzate secondo i parametri ImageNet. I pesi del Teacher non vengono aggiornati.

Nel modulo OCBE, la prima feature viene ridotta spazialmente con due convoluzioni 3×3 a stride 2, ciascuna seguita da Batch Normalization e ReLU; la seconda è ridotta con un solo blocco analogo. Le due mappe sono concatenate con la terza feature e trasformate da un blocco residuo corrispondente al livello profondo della Wide ResNet. Il decoder utilizza blocchi residui con convoluzioni trasposte. Non sono presenti collegamenti *skip* diretti tra Teacher e Student; le connessioni residue appartengono ai blocchi interni del decoder.

Il codice definisce anche una convoluzione 1×1 denominata `conv4`, che non è richiamata nel metodo `forward` della configurazione corrente e non viene pertanto considerata parte del percorso effettivo.

Addestramento di OCBE e decoder

L'addestramento usa batch da 16 immagini, due processi di caricamento e 200 epoche. L'ottimizzatore Adam aggiorna soltanto OCBE e Student Decoder con tasso di apprendimento 5×10^{-3} e coefficienti $\beta_1 = 0{,}5$, $\beta_2 = 0{,}999$.

Per ciascuno dei tre livelli, l'implementazione appiattisce tutte le dimensioni non appartenenti al batch, calcola la similarità coseno tra il vettore del Teacher e quello ricostruito e media sul batch. La perdita complessiva è la somma delle tre discrepanze:

$$\mathcal{L}_{\mathrm{RD}} = \sum_{k=1}^{3} \left(1 - \frac{1}{B} \sum_{b=1}^{B} \cos\Big(\mathrm{vec}(f_k^{(b)}), \mathrm{vec}(\widehat{f}_k^{(b)}) \Big) \right). \tag{3.6}$$

Inferenza e salvataggio degli artefatti

In inferenza, la similarità coseno è invece calcolata lungo i canali per ogni coordinata spaziale, come nell'Equazione 3.5. Le tre mappe sono interpolate bilinearmente alla dimensione originale, sommate e filtrate con una gaussiana di deviazione standard 4.

Il checkpoint PTH contiene i parametri di OCBE e Student Decoder insieme ai metadati necessari; il backbone congelato è ricostruito separatamente dai pesi Torchvision. La pipeline produce inoltre cronologia CSV, manifesto JSON e mappe TIFF a 32 bit.

3.6 FastFlow

3.6.1 Descrizione del modello

Normalizing Flow bidimensionali

FastFlow modella la distribuzione delle feature normali mediante flussi di normalizzazione bidimensionali [28]. In luogo di appiattire la rappresentazione, le trasformazioni operano su tensori con dimensioni di canale, altezza e larghezza, conservando la corrispondenza spaziale necessaria alla localizzazione.

Un backbone estrae feature a risoluzioni differenti. Per ciascuna scala, un flusso invertibile trasforma la feature x in una variabile latente z che segue una distribuzione normale standard. La densità nel dominio delle feature può essere calcolata esattamente mediante la formula del cambio di variabile.

Trasformazioni invertibili e verosimiglianza

Data una trasformazione invertibile $z = f_\theta(x)$, vale

$$\log p_X(x) = \log p_Z(f_\theta(x)) + \log \left| \det \frac{\partial f_\theta}{\partial x} \right| . \tag{3.7}$$

I blocchi di accoppiamento affine dividono i canali in due parti: una parte condiziona scala e traslazione applicate all'altra. Alternando suddivisioni e permutazioni, tutti i canali partecipano alle trasformazioni pur mantenendo calcolabili l'inversa e il determinante Jacobiano.

3.6.2 Implementazione nel progetto

Backbone e normalizzazione delle feature

L'implementazione è un porting locale derivato dal commit Anomalib `889a67f5`. Non richiede a runtime Lightning, Anomalib, timm o FrEIA: il backbone è fornito da Torchvision e le componenti necessarie dei flussi sono implementate nei moduli del progetto.

La configurazione supporta Wide ResNet-50-2 con pesi ImageNet-1K V1. Il backbone è congelato e produce le feature di `layer1`, `layer2` e `layer3`, aventi rispettivamente 256, 512 e 1024 canali e fattori di riduzione spaziale 4, 8 e 16. Le immagini sono ridimensionate a 256×256 e normalizzate con i parametri ImageNet. A ogni scala è applicata una Layer Normalization addestrabile sull'intero tensore (C, H, W).

La variante implementata non include backbone Transformer o Vision Transformer; tali possibilità appartengono alla formulazione generale del metodo, ma non alla configurazione del repository.

Blocchi di flusso e addestramento

Per ciascuna delle tre scale è costruito un `FastFlowBlock` composto da otto passi di accoppiamento invertibile. Poiché `Conv3x3Only` è disattivato, le sottoreti alternano kernel 3×3 e 1×1. Il rapporto dei canali nascosti è 1 e il parametro di *clamp* è 2.

Ogni passo esegue un accoppiamento affine, una trasformazione affine globale per canale e una permutazione deterministica inizializzata dal seme. La scala dell'accoppiamento è parametrizzata tramite una tangente iperbolica e un fattore 0,1; la scala globale usa una parametrizzazione *softplus*. Non è presente una fase separata di ActNorm dipendente dai dati.

L'addestramento usa batch da 8 immagini, due processi di caricamento e 200 epoche. Adam aggiorna Layer Normalization e flussi con tasso di apprendimento 10^{-3} e decadimento dei pesi 10^{-5}. Per ogni scala, la funzione obiettivo combina l'energia della variabile latente con il log-determinante Jacobiano:

$$\mathcal{L}_{\mathrm{FF}} = \sum_{k=1}^{3} \frac{1}{B} \sum_{b=1}^{B} \left(\frac{1}{2} \| z_k^{(b)} \|_2^2 - J_k^{(b)} \right), \tag{3.8}$$

dove $J_k^{(b)}$ è il log-determinante prodotto dal flusso alla scala k. I checkpoint intermedi sono salvati ogni cinque epoche e consentono la ripresa dell'esecuzione.

Inferenza e salvataggio degli artefatti

La predizione utilizza batch da 4 immagini. Per ogni scala, il codice calcola la media sui canali di $-\frac{1}{2}z^2$, la trasforma esponenzialmente e ne cambia il segno per ottenere la mappa locale. Le mappe sono interpolate a 256×256 e aggregate mediante media; la pipeline le riporta poi alla dimensione originale e applica un filtro gaussiano con deviazione standard 4.

I checkpoint PTH contengono i parametri delle normalizzazioni e dei flussi, mentre il backbone congelato viene ricaricato separatamente. Sono prodotti checkpoint intermedi e finali, cronologia CSV, manifesto JSON e mappe TIFF in virgola mobile a 32 bit.

3.7 Sintesi delle configurazioni implementative

La Tabella 3.2 riassume le configurazioni effettivamente selezionate. Essa descrive il ruolo di ImageNet e MVTec AD senza estendere le conclusioni oltre quanto registrato nel repository.

Modello	**Rappresentazione**	**Input e batch**	**Adattamento su MVTec AD**
CCW-SSIM	Autoencoder addestrato da zero	patch 128 × 128, batch 64	200 epoche, sole immagini normali
PaDiM	Wide ResNet-50-2 ImageNet congelata	224 × 224, batch 32	un passaggio per la stima statistica
RD4AD	Teacher ImageNet congelato, OCBE e decoder addestrabili	256 × 256, batch 16	200 epoche, sole immagini normali
FastFlow	Wide ResNet-50-2 ImageNet congelata e flussi addestrabili	256 × 256, batch 8	200 epoche, sole immagini normali

Tabella 3.2: Sintesi delle configurazioni implementative presenti nel repository.

Ne consegue che l'indicazione di ImageNet deve essere riferita unicamente al pre-addestramento dei backbone di PaDiM, RD4AD e FastFlow. Il dataset specifico della pipeline descritta nel capitolo è MVTec AD, nella categoria `leather`; per CCW-SSIM esso costituisce anche l'unica sorgente dei dati usati per apprendere i parametri del modello.

Bibliografia

[1] Abdelrahman Alzarooni, Ehtesham Iqbal, Samee Ullah Khan, Sajid Javed, Brain Moyo, and Yusra Abdulrahman. Anomaly detection for industrial applications, its challenges, solutions, and future directions: A review. *arXiv preprint arXiv:2501.11310*, 2024.

[2] Aimira Baitieva, David Hurych, Victor Besnier, and Olivier Bernard. Supervised anomaly detection for complex industrial images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17754–17762, 2024.

[3] Simone Berasi. Ccw-ssim: a novel color-enhanced structural similarity metric applied to autoencoders for anomaly detection. Master's thesis, Politecnico di Milano, 2023.

[4] Paul Bergmann, Kilian Batzner, Michael Fauser, David Sattlegger, and Carsten Steger. The mvtec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. *International Journal of Computer Vision*, 129(4):1038–1059, 2021.

[5] João P. C. Bertoldo, Dick Ameln, Ashwin Vaidya, and Samet Akçay. Aupimo: Redefining anomaly localization benchmarks with high speed and low tolerance. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2024.

[6] Yunkang Cao, Xiaohao Xu, Jiangning Zhang, Yuqi Cheng, Xiaonan Huang, Guansong Pang, and Weiming Shen. A survey on visual anomaly detection: Challenge, approach, and prospect. *arXiv preprint arXiv:2401.16402*, 2024.

[7] Yajie Cui, Zhaoxiang Liu, and Shiguo Lian. A survey on unsupervised anomaly detection algorithms for industrial images. *IEEE Access*, 11:55297–55315, 2023.

[8] Yajie Cui, Zhaoxiang Liu, and Shiguo Lian. A survey on unsupervised anomaly detection algorithms for industrial images. *IEEE Access*, 11:55297–55315, 2023.

[9] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. Padim: a patch distribution modeling framework for anomaly detection and localization. In *International Conference on Pattern Recognition (ICPR)*, pages 4790–4797, 2021.

[10] Hanqiu Deng and Xingyu Li. Anomaly detection via reverse distillation from one-class embedding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4477–4486, 2022.

[11] Kamil Faber, Christopher Kanan, Vincenzo Lomonaco, and Roberto Corizzo. Continual anomaly detection: A comprehensive survey and research roadmap. *Preprints.org*, 2026.

[12] David Guthrie, Louise Guthrie, Ben Allison, and Yorick Wilks. Unsupervised anomaly detection. In *Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1624–1628, 2007.

[13] Lars Heckler-Kram, Jan-Hendrik Neudeck, Ulla Scheler, Rebecca Konig, and Carsten Steger. The mvtec ad 2 dataset: Advanced scenarios for unsupervised anomaly detection. *arXiv preprint arXiv:2503.21622*, 2025.

[14] Lars Heckler-Kram, Jan-Hendrik Neudeck, Ulla Scheler, Rebecca König, and Carsten Steger. The mvtec ad 2 dataset: Advanced scenarios for unsupervised anomaly detection. *International Journal of Computer Vision*, 134(4):175–197, 2026.

[15] Tangqing Li, Zheng Wang, Siying Liu, and Wen-Yan Lin. Deep unsupervised anomaly detection. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 3636–3645, 2021.

[16] Zhuo Li, Yuhao Yan, Xiangheng Wang, Yifei Ge, and Lin Meng. A survey of deep learning for industrial visual anomaly detection. *Artificial Intelligence Review*, 58(9):279, 2025.

[17] Roya Morshedi and S. Mojtaba Matinkhah. A comprehensive review of deep learning techniques for anomaly detection in iot networks: Methods, challenges and datasets. *Engineering Reports*, 7(9):e70415, 2025.

[18] MVTec. The mvtec logical constraints anomaly detection dataset.

[19] G. Pang, C. Shen, L. Cao, and A. Van Den Hengel. Deep learning for anomaly detection: A review. *ACM Computing Surveys (CSUR)*, 54(2):1–38, 2021.

[20] Guansong Pang, Longbing Cao, and Charu Aggarwal. Deep learning for anomaly detection: Challenges, methods, and opportunities. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM)*, pages 1127–1130, 2021.

[21] Jing Ren, Feng Xia, Yemeng Liu, and Ivan Lee. Deep video anomaly detection: Opportunities and challenges. In *Proceedings of the International Conference on Data Mining Workshops (ICDMW)*, pages 959–966, 2021.

[22] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Scholkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14318–14328, 2022.

[23] Tung Minh Tran, Tu N. Vu, Nguyen D. Vo, Tam V. Nguyen, and Khang Nguyen. Anomaly analysis in images and videos: A comprehensive review. *ACM Computing Surveys*, 55(7):148:1–148:37, 2022.

[24] Miryam Elizabeth Villa-Pérez, Miguel Á. Álvarez-Carmona, Octavio Loyola-González, Miguel Angel Medina-Pérez, Juan Carlos Velazco-Rossell, and Kim-Kwang Raymond Choo. Semi-supervised anomaly detection algorithms: A comparative summary and future research directions. *Knowledge-Based Systems*, 218:106878, 2021.

[25] Y. Wan, Y. Lang, and L. Yao. Dcs: A zero-shot anomaly detection framework with dino-clip-sam integration. *Applied Sciences*, 16(4):1836, 2026.

[26] Jie Yang, Ruijie Xu, Zhiquan Qi, and Yong Shi. Visual anomaly detection for images: A survey. *arXiv preprint arXiv:2109.13157*, 2021.

[27] Min Yang and Jiajie Zhang. Data anomaly detection in the internet of things: A review of current trends and research challenges. *International Journal of Advanced Computer Science and Applications*, 14(9), 2023.

[28] Jiawei Yu, Ye Zheng, Xiang Wang, Wei Li, Yushuang Wu, Rui Zhao, and Liwei Wu. Fastflow: Unsupervised anomaly detection and localization via 2d normalizing flows. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[29] Ye Zheng, Xiang Wang, Yu Qi, Wei Li, and Liwei Wu. Benchmarking unsupervised anomaly detection and localization. *arXiv preprint arXiv:2205.14852*, 2022.